

Genova AI Agent Operating System

Bilan Complet du SaaS

Fonctionnalites, Architecture et Comportement des
Agents IA

Version 1.0 | Mai 2026

Next.js 16 + Prisma + PostgreSQL + TypeScript

Document de reference technique

Table des Matieres

1. Vue d'ensemble du SaaS Genova	4
2. Fonctionnalités principales	4
2.1 Authentification et Sécurité	5
2.2 Système d'Agents IA Autonomes	5
2.3 Système de Mémoire à 3 Niveaux	7
2.4 Registre d'Outils et Système de Permissions	8
2.5 Routeur IA Multi-Fournisseurs	8
2.6 Génération de Médias (Images et Vidéos)	9
2.7 Intégration WhatsApp	9
2.8 Workflows et Orchestration Multi-Agents	10
2.9 Gardes-fous et Validations	10
2.10 Base de Connaissances et RAG	11
3. Comportement détaillé des Agents IA en exécution	11
3.1 Cycle de vie d'une exécution	11
3.2 Mécanismes de décision et de réflexion	12
3.3 Autocorrection et adaptation	12
3.4 Apprentissage automatique en cours d'exécution	13
3.5 Gestion des états et reprise d'exécution	13
4. Architecture technique	14
4.1 Structure des API (40+ endpoints)	14
4.2 Base de données (31 tables PostgreSQL)	14
4.3 Streaming et observabilité	15
5. Services locaux et infrastructure	15
6. Interface utilisateur	16
7. Points d'attention et améliorations futures	17
7.1 Éléments fonctionnels mais nécessitant des améliorations	17

7.2 Améliorations architecturales recommandées	17
--	----

1. Vue d'ensemble du SaaS Genova

Genova est un systeme d'exploitation pour agents IA, concu comme une plateforme SaaS complete permettant de creer, gerer et orchestrer des agents intelligents autonomes. L'architecture repose sur Next.js 16 (App Router), Prisma ORM avec PostgreSQL, TypeScript strict, Tailwind CSS 4, shadcn/ui, et Framer Motion pour les animations. Le systeme offre une interface utilisateur single-page application (SPA) avec navigation par vues geree via Zustand, offrant une experience fluide et reactive.

La plateforme se distingue par son architecture a plusieurs niveaux : un moteur d'agents autonome avec boucle ReAct, un systeme de memoire a 3 niveaux (court terme, long terme, episodique), un routeur IA multi-fournisseurs avec basculement automatique, un pipeline de generation de medias (images et videos) avec chaines de secours, une integration WhatsApp avec Baileys et Cloud API, et un systeme de securite complet incluant RBAC, garde-fous, et journalisation d'audit.

L'infrastructure locale comprend PostgreSQL (port 5432) pour les 31 tables de la base de donnees, PocketBase (port 8090) pour le cache et les fichiers, n8n (port 5678) pour l'automatisation des workflows, ComfyUI (port 8188) pour la generation d'images locale, et une API video locale (port 8189) pour CogVideoX-2B et VideoCrafter2. Chaque service est gere par des scripts de demarrage et d'arret centralises.

Composant	Technologie	Port / Service
Frontend	Next.js 16 + React 19 + Tailwind CSS 4	Port 3000
Base de donnees	PostgreSQL + Prisma ORM (31 modeles)	Port 5432
Agents IA	ReAct Loop + StateGraph (LangGraph-style)	Interne
Routeur IA	Groq (P1) > OpenRouter (P2) > z-ai-sdk	API REST
Medias	ComfyUI + OpenRouter + Replicate + z-ai-sdk	Port 8188/8189
WhatsApp	Baileys (P1) > Cloud API v21.0 (fallback)	Webhook
Automatisation	n8n v2.22.5	Port 5678
Cache/Fichiers	PocketBase	Port 8090
Auth	PBKDF2 + httpOnly cookies + RBAC	Middleware

Tableau 1 : Stack technologique de Genova

2. Fonctionnalites principales

2.1 Authentification et Securite

Le systeme d'authentification de Genova utilise PBKDF2 avec un sel unique par utilisateur (32 octets, 100 000 iterations, SHA-512, cles de 64 octets). Les mots de passe sont stockes au format "pbkdf2:iterations:salt:hash" ou le sel est integre directement dans la chaine de hachage, eliminant le besoin d'un stockage separe. Le systeme supporte la migration automatique depuis 4 formats de hachage herites, effectuee de maniere transparente lors de chaque connexion reussie.

Les sessions sont gerees via des cookies httpOnly avec un token de session de 48 octets hexadecimaux. Chaque utilisateur peut avoir jusqu'a 10 sessions simultanees, les plus anciennes etant automatiquement expulsees. La duree de vie d'une session est de 24 heures, avec un token de rafraichissement valide 7 jours. Toutes les operations de session (creation, rafraichissement, expulsion, deconnexion) sont journalisees dans la table AuditLog pour conformite reglementaire.

Le controle d'accès basé sur les rôles (RBAC) définit trois niveaux hiérarchiques : utilisateur (niveau 0), administrateur (niveau 1), et super_administrateur (niveau 2). La fonction `hasRole()` effectue une vérification hiérarchique, permettant à un super_admin d'accéder aux ressources admin. Les en-têtes de sécurité incluent X-Frame-Options DENY, X-Content-Type-Options nosniff, Content-Security-Policy, et Permissions-Policy. Le middleware d'authentification protège toutes les routes `/api/*` avec une liste d'exemption pour les routes publiques.

2.2 Systeme d'Agents IA Autonomes

Le cœur de Genova réside dans son système d'agents IA autonomes, reposant sur deux modes d'exécution distincts mais complémentaires : la boucle ReAct classique et le graphe d'état de style LangGraph. Ces deux modes partagent les mêmes types de contexte d'exécution (`ExecutionContext`), d'étapes (`ExecutionStep`), et de plan d'exécution (`ExecutionPlan`), garantissant une cohérence architecturale fondamentale.

Boucle ReAct : Penser -> Agir -> Observer -> Reflechir -> Reessayer

La boucle ReAct constitue le mode d'exécution original et le plus éprouvé. L'agent suit un cycle rigoureux en cinq phases. Pendant la phase THINK, l'agent raisonne sur la situation actuelle, analyse les options disponibles, et produit une décision structurée en JSON incluant sa pensée, l'action choisie, les paramètres d'entrée, un indicateur de finalité, et un score de confiance. Lors de la phase ACT, l'outil sélectionné est exécuté via le ToolRegistry avec un pipeline complet : vérification des permissions, validation des capacités, application des politiques d'exécution, validation des paramètres, exécution en bac à sable, et analyse du

resultat.

La phase OBSERVE capture le resultat de l'action sous forme d'etape d'observation. Vient ensuite la phase REFLECT, veritable differentiateur de Genova : l'agent evalue sa propre progression en calculant un score de progression (0-1), un score de qualite, une indication de necessite de reessai, et une recommandation parmi cinq options possibles (continuer, reessayer, adapter, arreter, repondre). Enfin, la phase RETRY permet l'autocorrection : si la reflexion recommande un reessai, l'agent analyse l'erreur, propose une approche alternative, et tente une nouvelle execution corrgee.

Le systeme integre des mecanismes de securite robustes : limite maximale de 10 etapes par execution, arret automatique apres 3 erreurs consecutives, reflexion forcee a 70% du budget d'etapes sans progression, verification des garde-fous pour les outils dangereux, et persistance de l'etat d'execution dans la table AgentExecution pour reprise ulterieure.

Graphe d'Etat : Mode LangGraph

Le deuxieme mode d'execution implmente un graphe d'etat de style LangGraph avec 11 phases distinctes : INIT, PLAN, THINK, ACT, OBSERVE, REFLECT, CORRECT, RETRY, RESPOND, ERROR, et COMPLETE. Chaque phase est un noeud du graphe avec un gestionnaire dedie, et les transitions entre noeuds peuvent etre conditionnelles.

Le systeme de detection de cycles limite les visites par noeud a 5 (10 pour THINK, REFLECT, OBSERVE) et le total d'iterations a 50, prevenant les boucles infinies. La persistance d'etat via StatePersistence permet de sauvegarder et reprendre l'execution d'un graphe a tout moment, avec restauration complete du detecteur de cycles. Le graphe peut etre exporte au format DOT de Graphviz pour visualisation.

Type d'Agent	Outils Autorises	Cas d'Usage
Ventes (sales)	web_search, database_query, calculator	Prospection, devis, suivi commercial
Support (support)	database_query, web_search	Assistance client, resolution de problemes
Marketing (marketing)	web_search, calculator, database_query	Campagnes, analyse de marche, contenu
Recherche (research)	web_search, database_query, filesystem	Veille, analyses, rapports
RH (rh)	database_query, calculator	Gestion du personnel, paie, conges

Comptabilite (accounting)	calculator, database_query	Factures, bilans, declarations
Personnalise (custom)	Tous les outils	Configuration libre par l'utilisateur

Tableau 2 : Types d'agents et leurs capacites

2.3 Systeme de Memoire a 3 Niveaux

Le systeme de memoire de Genova opere sur trois niveaux distincts et complementaires, chacun avec un role specifique dans la capacite de l'agent a apprendre et a s'adapter au fil du temps.

La memoire a court terme gere le contexte de conversation avec une sensibilite au budget de tokens. Elle utilise un mecanisme de fenetre glissante qui conserve les N derniers messages, avec une auto-summarisation lorsque le contexte depasse la capacite. Une estimation de l'importance est effectuee pour chaque message afin de prioriser les informations les plus pertinentes lors de la selection.

La memoire a long terme constitue le pilier de l'apprentissage persistant. Elle utilise une recherche hybride combinant la recherche semantique par vecteurs (embeddings) et la recherche par mots-cles (BM25/TF-IDF), fusionnees via le Reciprocal Rank Fusion (RRF). Le score de pertinence composite combine quatre facteurs ponderes : recence (25%), frequence (20%), importance (25%), et pertinence semantique (30%). Les categories de memoire incluent les preferences, projets, documents, contexte de workflow, apprentissages d'agents, memoires episodiques, et memoires semantiques.

La memoire episodique enregistre les experiences vecues par l'agent avec une valence emotionnelle (positive ou negative) et des lecons apprises. Les experiences negatives (echecs) recoivent un bonus d'importance de +0.1 car elles representent des lecons critiques. Le systeme de compression semantique groupe les memoires similaires (seuil de similarite 0.7) et les condense via LLM, eliminant les redondances tout en preservant les informations cles. L'elagage automatique supprime les memoires de faible importance apres 90 jours, base sur le score composite.

L'apprentissage automatique est un mecanisme fondamental : apres chaque execution d'agent, les observations reussies (confiance > 0.7) sont automatiquement stockees dans la memoire a long terme avec la categorie "agent_learning" et des tags incluant le type d'agent et l'outil utilise. Ce mecanisme permet aux agents de s'ameliorer continuellement au fil de leurs interactions.

2.4 Registre d'Outils et Systeme de Permissions

Le ToolRegistry est le coeur du systeme d'execution des outils, implementant un pipeline a 7 etapes pour chaque appel : Permission Check, Capability Check, Execution Policy Check, Parameter Validation, Sandbox Check, Execution, et Result Parsing. Ce pipeline garantit que chaque execution d'outil est securisee, validee, et observable.

Le systeme de permissions opere sur 5 couches distinctes. La Permission Layer gere les listes d'outils autorises/interdits par utilisateur, avec une limite de 5 appels dangereux par session et des exigences d'approbation pour les outils sensibles. Le Capability Manager accorde des capacites granulaires par agent avec des actions, portees, limites d'appels, restrictions temporelles, et dates d'expiration. L'Execution Policy Manager applique des politiques regissant les types d'actions (autoriser, interdire, limiter le debit, exiger une approbation, restreindre temporellement, limiter les ressources). Le Tool-Scoped Auth fournit des tokens d'authentification OAuth-like par outil et par agent. Enfin, l'Execution Sandbox isole chaque execution avec un timeout de 30 secondes par default.

Les outils disponibles incluent la recherche web (via z-ai-web-dev-sdk), le calcul mathematique, les requetes de base de donnees, les operations sur le systeme de fichiers (dangereux), et l'execution de code JavaScript en bac a sable. L'executeur de code bloque les appels require, import, process, fs, fetch, et eval, tout en fournissant des globals sures comme Math, JSON, et Array. Le Result Parser sanitize les sorties en masquant les cles API (sk-*, gsk_*), les tokens Bearer, et les chemins de fichiers sensibles.

2.5 Routeur IA Multi-Fournisseurs

Le routeur IA d'Genova implemente une chaine de priorite avec basculement automatique entre trois fournisseurs. Groq occupe la priorite 1 avec les modeles llama-3.3-70b-versatile (default), llama-3.1-8b-instant (rapide), et llama-3.3-70b-versatile (puissant). OpenRouter constitue la priorite 2 avec meta-llama/llama-3.1-8b-instruct:free (default et rapide) et meta-llama/llama-3.1-70b-instruct (puissant). Le z-ai-web-dev-sdk sert de fallback universel lorsque aucune cle API directe n'est disponible.

Le systeme de retry utilise un backoff exponentiel avec un delai initial de 500ms, un maximum de 3 tentatives par fournisseur, et un timeout de 60 secondes. Les erreurs transitoires (429, 5xx, timeout, reseau) declenchent un retry, tandis que les erreurs non-transitoires provoquent un basculement immediat vers le fournisseur suivant. L'estimation des couts est calculee en temps reel avec les tarifs actuels de

chaque fournisseur, et chaque appel est trace dans la table AICost avec le nombre detokens, le cout en USD, et l'identifiant de requete.

Le routeur supporte le streaming via async generators, permettant l'affichage en temps reel des reponses dans l'interface utilisateur. La fonction chatCompletion() offre une interface simplifiee avec des modes predefinis (default, fast, powerful, quick_chat, analysis, reasoning, orchestration) qui mappent automatiquement vers le tier de modele approprié.

2.6 Generation de Medias (Images et Videos)

Le pipeline de generation d'images implemente une chaine de secours a 2 niveaux avec validation et limitation du debit. Le niveau 1 utilise OpenRouter avec les modeles gratuits Flux-1-schnell-free et Stable-Diffusion-XL-free. Le niveau 2 bascule vers le z-ai-web-dev-sdk. Le systeme impose une limite de 10 images par heure par utilisateur, avec validation du prompt (suppression HTML, limitation a 2000 caracteres) et des dimensions (512/768/1024/1344). Chaque generation est tracee dans les tables ImageGeneration et AICost.

Le pipeline de generation de videos est plus elabore avec une chaine de secours a 3 niveaux. Le niveau 1 tente l'API locale avec CogVideoX-2B (720x480, 49 frames, 8fps) ou VideoCrafter2 (512x320, 16 frames, 28fps) via un health check prealable. Le niveau 2 bascule vers Replicate Cloud API avec un polling de completion (intervalle 5s, maximum 120 tentatives, timeout 10 minutes). Le niveau 3 signale l'indisponibilite si tous les fournisseurs echouent. La limite est de 5 videos par heure, avec des couts de 0 USD en local et 0.05 USD dans le cloud.

2.7 Integration WhatsApp

L'integration WhatsApp repose sur une architecture a double canal avec basculement automatique. Le canal principal utilise Baileys (WhatsApp Web library) pour une connectivite directe sans frais, avec gestion des QR codes, sessions persistantes, et reconnexion automatique. Le canal de secours utilise l'API WhatsApp Cloud v21.0 officielle de Meta, activee automatiquement apres 3 echecs consecutifs de Baileys, avec une tentative de retour a Baileys apres 5 minutes de recuperation.

Le pipeline de reponse automatique implemente un delai configurable de 10 secondes entre la reception d'un message client et la reponse de l'agent IA, simulant un comportement humain naturel. Le flux est : onMessage (evenement de reception) > attente de RESPONSE_DELAY_MS (10 000ms) > generation de la reponse via l'IA > envoi via sendMessage. Le webhook WhatsApp Cloud API gere la verification GET (hub.mode + hub.verify_token) et la reception POST des messages entrants.

La securite WhatsApp inclut la sanitization des messages (suppression HTML, limitation a 4096 caracteres), la validation des numeros de telephone au format E.164, et le retry avec backoff exponentiel en cas d'echec d'envoi. La fonction `sendImage()` inclus une degradation gracieuse vers l'envoi de texte si l'image echoue.

2.8 Workflows et Orchestration Multi-Agents

Le systeme de workflows permet de definir des sequences d'actions automatisees avec des declencheurs, des etapes, et un suivi de progression. Chaque workflow possede un statut (brouillon, actif, en pause, termine, echoue), un index de tache courant, et des etapes stockees en JSON. Les workflows sont orquestes via l'API `/api/workflows` avec des endpoints de creation, lecture, mise a jour, suppression, et execution.

L'orchestration multi-agents utilise le planificateur (Planner) pour decomposer un objectif complexe en sous-taches assignees a des agents specifiques avec gestion des dependances. L'AgentManager coordonne l'execution des plans multi-agents avec resolution des dependances : les taches dont les dependances sont satisfaites sont executees en premier, et les resultats des taches precedentes sont injectes comme contexte dans les taches dependantes. L'evalateur de plan (`evaluatePlanProgress`) determine si un plan necessite des ajustements en fonction des taches reussies et echouees.

2.9 Gardes-fous et Validations

Le systeme de garde-fous (Guardrails) permet de definir des regles de securite que les agents doivent respecter pendant leur execution. Chaque garde-fou possede un nom, un type, une description, des regles (JSON), une severite (info, avertissement, critique), et un statut actif/inactif. Les regles peuvent bloquer des outils dangereux, interdire certaines actions, ou exiger une approbation humaine avant execution.

Lorsqu'un agent tente d'executer un outil dangereux alors que des garde-fous sont actifs, le systeme verifie chaque garde-fou actif pour determiner si l'action doit etre bloquee. Si un garde-fou bloque l'action, l'agent passe en statut `"awaiting_approval"` et une demande d'approbation est creee dans la table `ApprovalRequest`. L'utilisateur peut ensuite approuver ou rejeter la demande via l'interface ou l'API.

Le validateur de prompts (`PromptValidator`) detecte les tentatives d'injection dans les parametres des outils, avec un niveau de menace (low, medium, high) et une liste de risques identifies. Les parametres avec un niveau de menace "high" sont

automatiquement rejetes, prevenant les attaques par injection de prompts.

2.10 Base de Connaissances et RAG

Le systeme RAG (Retrieval-Augmented Generation) de Genova combine un retriever hybride avec un stockage vectoriel persistant. Le retriever utilise trois strategies de recherche : VectorStore (recherche semantique par embeddings), BM25 (recherche par mots-cles avec scoring probabiliste), et la fusion RRF (Reciprocal Rank Fusion) pour combiner les resultats. Le traitement des documents inclus le decoupage automatique en chunks, la generation d'embeddings, et le stockage dans les tables Document et DocumentChunk.

L'augmentation de prompts integre les resultats de recherche directement dans le contexte de l'agent, avec des citations de sources permettant la tracabilite des informations. L'extracteur de connaissances analyse les conversations terminees et extrait automatiquement les informations cles, decisions, et lecons apprises via LLM, les stockant dans la table Knowledge avec des categories et scores d'importance.

3. Comportement detaille des Agents IA en execution

Cette section decrit en detail le comportement observable des agents IA pendant leur temps de fonctionnement, depuis l'initialisation jusqu'a la completion ou l'echec, en passant par les mecanismes d'adaptation et d'apprentissage qui definissent leur autonomie.

3.1 Cycle de vie d'une execution

Lorsqu'un utilisateur soumet une tache a un agent, le systeme cree un ExecutionContext complet incluant l'identite de l'agent, la tache, les outils disponibles, et la configuration. L'execution demarre par le chargement asynchrone de la configuration agent depuis la base de donnees, du contexte de memoire a long terme pertinent via recherche hybride, et de la memoire a court terme depuis la conversation existante si applicable.

Un plan d'execution initial est ensuite genere par LLM avec 3 a 5 etapes maximum, chacune avec une description, un indice d'outil suggere, et des dependances optionnelles. Si la generation du plan echoue, un plan par default a 3 etapes est utilise (analyser, executer, verifier). Ce plan est adaptatif : la reflexion de l'agent peut modifier le plan en cours d'execution, chaque adaptation etant enregistree dans l'historique du plan.

Pendant l'exécution, chaque étape est tracée par le Tracer avec des métriques détaillées : type d'étape, contenu, durée, tokens utilisés, modèle et fournisseur. Les étapes sont transmises en temps réel via un callback `onStep()` pour l'affichage dans l'interface utilisateur. L'état global de l'exécution est périodiquement persisté dans la base de données pour permettre la reprise en cas d'interruption.

3.2 Mécanismes de décision et de réflexion

La phase THINK constitue le point de décision de l'agent. Le prompt système inclus la mission, la configuration, la mémoire à long terme, les outils disponibles, le contexte du plan, et l'historique des 12 dernières étapes. L'agent doit répondre en JSON valide avec cinq champs : `thought` (raisonnement), `action` (outil ou "respond"), `actionInput` (paramètres), `isFinal` (booléen), et `confidence` (0-1). Si le JSON est invalide, un fallback traite le contenu comme une réponse finale avec confiance 0.5.

La phase REFLECT est le mécanisme d'autoévaluation le plus sophistiqué du système. L'agent évalue les 5 dernières étapes et produit une analyse structurée incluant : `progressScore` (0-1), `qualityScore` (0-1), `needsRetry` (booléen), `needsAdaptation` (booléen), `reflection` (analyse narrative), `recommendation` (continuer/retry/adapt/stop/respond), `alternativeApproach` (description), et `confidenceInResult` (0-1). Si le parsing échoue, le système effectue une évaluation simplifiée basée sur la présence d'erreurs dans les étapes récentes.

Les recommandations de réflexion déclenchent des actions spécifiques : "retry" active le mécanisme d'autocorrection avec une approche alternative, "adapt" modifie le plan d'exécution, "respond" fournit la réponse finale si la confiance est suffisante, "stop" arrête l'exécution avec un rapport d'échec, et "continuer" poursuit la boucle normalement. Ce système permet à l'agent de s'auto-évaluer objectivement et de prendre des décisions éclairées sur la poursuite ou non de ses actions.

3.3 Autocorrection et adaptation

Le mécanisme d'autocorrection (`retryWithCorrection`) se déclenche lorsque la réflexion recommande un réessai. L'agent analyse l'erreur rencontrée, la dernière action tentée, et l'approche alternative suggérée pour produire une action corrigée. Le prompt de correction demande à l'agent d'analyser pourquoi l'action a échoué, de proposer une action corrigée avec de nouveaux paramètres, et d'indiquer si la correction est finale.

Le nombre de tentatives de correction est limité à `maxRetries` (3 par défaut). Lorsque la limite est atteinte, l'exécution se termine avec un statut "failed" et un rapport détaillé de la dernière erreur. Chaque tentative est enregistrée comme étape.

de type "retry" avec le compteur de tentatives, permettant une tracabilite complete du processus d'autocorrection dans l'historique d'execution.

L'adaptation du plan (adaptPlan) se declenche lorsque la reflexion recommande une adaptation. L'etape courante du plan est marquee comme echouee, et l'historique des adaptations enregistre la raison, le plan original, et le plan adapte. L'agent peut ensuite reprendre la boucle THINK avec un contexte mis a jour incluant l'echec et la nouvelle direction suggeree.

3.4 Apprentissage automatique en cours d'execution

L'apprentissage automatique se produit a deux moments critiques du cycle de vie de l'agent. Premierement, pendant l'execution : chaque observation reussie (confiance > 0.7) est stockee dans la memoire a long terme comme entree de categorie "agent_learning" avec des tags incluant le type d'agent et l'outil utilise. Ce mecanisme permet a l'agent d'accumuler des connaissances procedurales au fil de ses executions.

Deuxiemement, apres l'execution : la fonction saveExecution() extrait les observations reussies de l'execution terminee et les stocke comme apprentissages dans la memoire a long terme. Chaque apprentissage est formate comme "Apprentissage de [nom_agent] : [contenu]" et recoit un score de pertinence egal a la confiance de l'observation. Ces apprentissages sont ensuite disponibles pour les futures executions du meme agent ou d'agents similaires via la recherche hybride.

L'extraction de connaissances depuis les conversations (extractAndStore) represente un troisieme canal d'apprentissage. Lorsqu'une conversation se termine, le systeme utilise le LLM pour extraire les informations cles, decisions, et lecons apprises, puis les stocke dans la base de connaissances avec des categories et scores d'importance. Si l'extraction par LLM echoue, un fallback base sur l'extraction de mots-cles est utilise pour garantir qu'aucune connaissance n'est perdue.

3.5 Gestion des etats et reprise d'execution

Le systeme de persistance d'etat permet de sauvegarder l'etat complet d'une execution en cours et de la reprendre ulterieurement. La fonction saveExecutionState() utilise upsert dans la table AgentExecution, stockant les etapes en JSON, le statut, la duree totale, les tokens utilises, et le cout estime. La fonction loadExecutionState() recharge un etat paused et reconstruit le contexte d'execution complet.

Pour le mode StateGraph, StatePersistence.save() enregistre l'etat du graphe, le noeud courant, et le resume du detecteur de cycles dans un objet

PersistedGraphState. La reprise (resume) restaure le detecteur de cycles avec lescomptes de visites precedents et reprend l'execution a partir du noeud suivant lenoeud sauvegarde, garantissant la continuite du comportement de l'agent memeapres une interruption.

4. Architecture technique

4.1 Structure des API (40+ endpoints)

L'API de Genova expose plus de 40 endpoints organises en modules fonctionnels. Le module Auth (9 endpoints) gere l'inscription, la connexion, la deconnexion, la verification email, et la gestion des sessions. Le module Agents (8 endpoints) permet la creation, mise a jour, suppression, chat (SSE streaming), execution, memoire, permissions, et basculement des agents. Le module AI (3 endpoints) offre le chat direct, l'orchestration multi-agents, et la validation de prompts.

Le module Workflows (4 endpoints) gere le cycle de vie complet des workflows. Le module WhatsApp (4 endpoints) gere l'envoi, la verification, la configuration, et les appels. Le module Social (3 endpoints) gere les comptes et publications. Les modules Images et Videos (4 endpoints) gerent la generation et le suivi des medias. Le module RAG (2 endpoints) gere le telechargement de documents et les requetes de connaissances.

Les modules Analytics (4 endpoints) fournissent les statistiques d'utilisation, de performance des agents, de couts IA, et de monitoring systeme. Le module Admin (2 endpoints) gere la blocklist d'URLs et les journaux d'audit. Les modules supplementaires couvrent les conversations, activites, garde-fous, approbations, taches, ressources, file d'attente, et dashboard.

4.2 Base de donnees (31 tables PostgreSQL)

La base de donnees PostgreSQL de Genova contient 31 tables synchronisees via Prisma, organises en domaines fonctionnels. Le domaine Core (4 tables) comprend les utilisateurs, sessions, reinitialisations de mot de passe, et verifications d'email. Le domaine Agents (6 tables) couvre les agents, permissions, journaux d'actions, memoires, executions, et sessions navigateur.

Le domaine Workflows (4 tables) inclut les workflows, taches, validations, et garde-fous. Le domaine Communication (3 tables) gere les conversations, messages, et configurations WhatsApp. Le domaine Knowledge (3 tables) couvre les entrees de connaissance, documents, et chunks de documents. Le domaine Media (2 tables) trace les generations d'images et de videos.

Le domaine Analytics (4 tables) comprend l'utilisation des agents, les couts IA, l'usage quotidien, et les evenements de monitoring. Le domaine Security (2 tables) couvre les journaux d'audit et la blocklist d'URLs. Le domaine Social (2 tables) gere les comptes sociaux. Le domaine Resources (1 table) stocke les ressources utilisateur. Des index strategiques sont places sur les champs frequemment requetes (role, dates, categories, severites) pour optimiser les performances.

Domaine	Tables	Comp tage
Core	users, sessions, password_resets, email_verifications	4
Agents	agents, agent_permissions, agent_action_logs, agent_memories, agent_executions, browser_sessions	6
Workflows	workflows, tasks, validations, guardrails	4
Communication	conversations, messages, whatsapp_configs	3
Knowledge	knowledge, documents, document_chunks	3
Media	image_generations, video_generations	2
Analytics	agent_usage, ai_costs, usage_daily, monitoring_events	4
Security	audit_logs, url_blocklist, approval_requests	3
Social	social_accounts, user_resources	2

Tableau 3 : Domaines fonctionnels de la base de donnees

4.3 Streaming et observabilite

Le systeme de streaming SSE (Server-Sent Events) gere la diffusion en temps reel des reponses des agents. Le StreamManager inclus un TokenBuffer pour le batch des tokens, un EventBatcher pour l'optimisation de l'envoi, un ProgressTracker pour le suivi de progression, et un mecanisme de backpressure pour eviter la surcharge. Le systeme de heartbeat maintient les connexions actives et detecte les deconnexions.

Le WebSocket manager complement le SSE pour les evenements en temps reel du flux agent (nouvelles etapes, changements de statut, demandes d'approbation). L'observabilite est assuree par le Tracer qui suit chaque execution d'agent avec un identifiant de trace unique, des metriques par etape, et une aggregation des statistiques (duree moyenne, cout, taux d'erreur).

5. Services locaux et infrastructure

Genova s'appuie sur une infrastructure locale complete pour assurer l'autonomie et la performance du systeme. Chaque service est installe et configure localement sur le serveur, reduisant la dependance aux services cloud externes et ameliorant la latence.

Service	Port	Role	Statut
PostgreSQL	5432	Base de donnees principale (31 tables)	Operationnel
PocketBase	8090	Cache, fichiers, donnees d'apprentissage	Operationnel
n8n	5678	Automatisation de workflows	Operationnel (v2.22.5)
ComfyUI	8188	Generation d'images locale	Necessite checkpoint
API Video	8189	CogVideoX-2B / VideoCrafter2	Configure
Next.js	3000	Application SaaS	Operationnel
Caddy	81	Reverse proxy	Configure

Tableau 4 : Infrastructure locale de Genova

Les scripts de gestion centralises (start-all.sh et stop-all.sh) permettent de demarrer et arreter tous les services en une seule commande. Le demarrage suit un ordre precis pour gerer les dependances : PostgreSQL en premier, puis PocketBase, n8n, ComfyUI, et enfin l'application Next.js. Chaque service est lance en arriere-plan avec verification de sante.

6. Interface utilisateur

L'interface utilisateur de Genova est une single-page application (SPA) construite avec React 19 et Tailwind CSS 4, offrant 10 vues principales accessibles via une navigation laterale geree par Zustand. Le layout comprend un en-tete (AppHeader) et une barre laterale (AppSidebar) avec navigation par icones et labels.

Les vues disponibles incluent le Dashboard (statistiques globales, activite recente, alertes), les Agents (liste, creation, detail avec chat en temps reel, monitoring d'execution), l'Automatisation (gestion des workflows), les Gardes-fous (configuration des regles de securite), la Coordination (orchestration multi-agents, constructeur visuel de workflows), les Parametres (configuration generale, approbations), les Analyses (tableaux de bord avec graphiques), les Medias (galerie de generation d'images et videos), et les Connaissances (gestion de la base de connaissances).

La bibliotheque de composants UI comprend plus de 40 composants shadcn/ui (accordion, alert, avatar, badge, button, card, dialog, dropdown-menu, form, input,

select, table, tabs, toast, tooltip, etc.) assurant une coherence visuelle a travers toutel'application. Les animations sont gerees par Framer Motion pour des transitionsfluides entre les vues et les etats des composants.

7. Points d'attention et ameliorations futures

Malgre la solidite architecturale de Genova, plusieurs points meritent une attention particuliere pour ameliorer la robustesse et la securite du systeme en production.

7.1 Elements fonctionnels mais necessitant des ameliorations

ComfyUI necessite le telechargement d'un checkpoint de modele pour la generation d'images locale. Actuellement, le systeme bascule vers OpenRouter et z-ai-sdk, mais l'activation de ComfyUI permettrait une generation d'images gratuite et rapide en local. Baileys requiert le scan d'un QR code pour connecter WhatsApp Web, une operation manuelle necessaire avant chaque nouvelle session. n8n necessite la generation et la configuration d'une cle API dans le fichier .env pour l'integration programmatique.

Le systeme d'authentification pourrait beneficier de refresh tokens plus robustes avec rotation automatique, d'un durcissement RBAC avec permissions granulaires par ressource, et d'un systeme d'audit plus complet avec alertes en temps reel. La base de donnees SQLite utilisee en developpement devrait etre migree vers PostgreSQL en production pour une meilleure concurrence et scalabilite.

7.2 Ameliorations architecturales recommandees

L'ajout d'un systeme de file d'attente persistant (Redis/Bull) pour les taches asynchrones ameliorerait la fiabilite du systeme de jobs. L'implementation d'un systeme de cache distribue (Redis) pour les sessions et les donnees frequemment accedees reduirait la charge sur PostgreSQL. L'ajout de tests unitaires et d'integration couvrant les chemins critiques (auth, execution d'agents, generation de medias) garantirait la non-regression lors des evolutions.

Le monitoring pourrait etre enrichi avec des metriques Prometheus/Grafana pour la surveillance en temps reel des performances. L'ajout d'un systeme de rate limiting persistant (au lieu de en-memoire) partagerait les limites entre les instances. Enfin, l'implementation d'un systeme de versionnage des agents permettrait de revenir a une configuration anterieure en cas de probleme.